

💡 ما هي المصفوفة؟

المصفوفة هي مجموعة من القيم من نفس النوع (مثل أعداد صحيحة أو نصوص) نخزنها تحت اسم واحد، ونصل لكل عنصر فيها باستخدام رقم ترتيبه داخل المصفوفة، والذي يُسمى **مؤشر (Index)** ويبدأ دائماً من الصفر.

مثال:

```
int[] grades = new int[5];
```

المصفوفة grades تحتوي على 5 أماكن (أو عناصر)، وكل عنصر يمكننا الوصول إليه بواسطة:

```
grades[0], grades[1], ..., grades[4]
```

📦 لماذا نستخدم المصفوفات؟

تخيل أنك تريد تخزين علامات 100 طالب.
بدلاً من تعريف 100 متغير:

```
int grade1, grade2, ..., grade100;
```

نستخدم مصفوفة:

```
int[] grades = new int[100];
```

وبالتالي نستطيع التكرار على كل العلامات باستخدام حلقة for.

🧠 كيفية تخزين المصفوفة في ذاكرة الحاسوب

- المتغيرات العادية (مثل `int a = 5;`) تُخزن في ذاكرة **stack**.
- المصفوفات تُخزن في ذاكرة **heap** لأنها كائنات (Objects).
- العنوان الذي يشير إلى المصفوفة يُخزن في **stack**.

🔗 طرق تعريف المصفوفة

1. تعريف بدون قيم أولية:

```
int[] numbers = new int[3]; // جميع القيم ستكون 0
```

2. تعريف مع إعطاء قيم:

```
int[] numbers = { 4, 9, 2 };
```

الوصول لعناصر المصفوفة 🗂️

يمكنك قراءة أو تعديل عنصر داخل المصفوفة باستخدام المؤشر:

```
Console.WriteLine(numbers[0]); // طباعة العنصر الأول
```

```
numbers[1] = 10; // تعديل العنصر الثاني
```

⚠️ لا يمكن استخدام مؤشر خارج حدود المصفوفة، مثل:

```
numbers[3] = 5; // خطأ إذا كانت المصفوفة حجمها 3 ❌
```

خاصية الطول (Length) 📏

تعيد عدد العناصر داخل المصفوفة:

```
Console.WriteLine(numbers.Length); // كم عنصر يوجد
```

حلقات التكرار والمصفوفات 🔄

عادة نستخدم for للمرور على جميع العناصر:

```
for (int i = 0; i < numbers.Length; i++)  
{  
    Console.WriteLine(numbers[i]);  
}
```

❌ الخروج عن حدود المصفوفة

إذا حاولت الوصول إلى مؤشر غير موجود (أكبر أو أصغر من المدى)، يظهر خطأ تنفيذ:

```
int[] a = new int[5];
```

هذا خاطئ لأن آخر فهرس هو 4 ❌ `a[5] = 10;`

🌀 نسخ مصفوفة أو تمريرها لدالة

✓ بالمرجع: (By Reference)

عند تمرير المصفوفة لدالة أو متغير آخر، فإنك تمرر عنوانها في الذاكرة وليس نسخة منها.

🔪 أي تعديل داخل الدالة يؤثر على المصفوفة الأصلية.

📌 عمليات شائعة على المصفوفات

✅ إيجاد الحد الأقصى:

```
int max = arr[0];
for (int i = 1; i < arr.Length; i++)
{
    if (arr[i] > max)
        max = arr[i];
}
```

✅ إيجاد المؤشر للحد الأقصى:

```
int index = 0;
for (int i = 1; i < arr.Length; i++)
{
    if (arr[i] > arr[index])
        index = i;
}
```

✅ طباعة المصفوفة بالعكس:

```
for (int i = arr.Length - 1; i >= 0; i--)
{
    Console.WriteLine(arr[i]);
}
```

🔗 المصفوفة المتناظرة (Palindrome)

هي مصفوفة إذا قرأتها من اليمين لليسار أو بالعكس، تحصل على نفس الترتيب.

✅ فحص:

```
bool isPal = true;
for (int i = 0; i < arr.Length / 2; i++)
```

```
{
    if (arr[i] != arr[arr.Length - 1 - i])
        isPal = false;
}
```

تدوير المصفوفة لليمين

نقل كل عنصر إلى اليمين، وآخر عنصر يصبح الأول.

مصفوفة عدادات (Counting Array)

نستخدمها لحساب كم مرة تكررت كل قيمة. مثلاً:

- عدد الطلاب الذين حصلوا على كل علامة.
- عدد المرات التي ظهرت بها أحرف معينة.

تمارين وأمثلة تطبيقية

1. طباعة العناصر الزوجية فقط:

```
for (int i = 0; i < arr.Length; i++)
    if (arr[i] % 2 == 0)
        Console.WriteLine(arr[i]);
```

2. حساب المعدل:

```
int sum = 0;
for (int i = 0; i < arr.Length; i++)
    sum += arr[i];
double avg = (double)sum / arr.Length;
```

3. إيجاد نقطة التوازن:

```
for (int i = 0; i < arr.Length; i++)
{
```

```

int left = 0, right = 0;

for (int j = 0; j < i; j++) left += arr[j];

for (int j = i + 1; j < arr.Length; j++) right += arr[j];

if (left == right)
{
    Console.WriteLine("نقطة التوازن في المؤشر " + i);
    break;
}
}

```

نصائح وأخطاء شائعة

الخطأ	التصحيح
استخدام <code>arr.Length</code> في حلقة <code>for</code> <code><=</code>	يجب استخدام <code>arr.Length</code> <code><</code> <code>i</code>
استخدام مصفوفة بدون إعطاؤها حجم	يجب تعريف الحجم أو القيم
التعديل على مصفوفة بدون التأكد من المؤشر	تحقق أن المؤشر ضمن النطاق